
◆ 1. RDD vs. DataFrame APIs

🔍 Summary:

- **RDD (Resilient Distributed Dataset):** Low-level, functional API for distributed data processing.
 - **DataFrame:** High-level abstraction over RDDs with **schema** and **SQL-like** operations.
-

✅ Example:

```
# RDD example
rdd = sc.parallelize([1, 2, 3, 4])
rdd_sum = rdd.reduce(lambda a, b: a + b)

# DataFrame example
df = spark.createDataFrame([(1, 'Alice'), (2, 'Bob')], ['id', 'name'])
df.filter(df.id > 1).show()
```

💬 Interview Q&A:

Q1: Why prefer DataFrame over RDD? A1:

- Optimization via Catalyst
 - Easier syntax
 - Better integration with Spark SQL and MLlib
-

Q2: When would you use RDD over DataFrame? A2:

- Custom or low-level transformations
 - Type-safe and complex object manipulations
-

Q3: What is a Dataset in Spark? A3: A Dataset is like a **typed DataFrame** (available in Scala/Java, not PySpark).

◆ 2. Lazy Evaluation

Summary:

In Spark, **transformations** are lazy — they're not executed until an **action** (like `.collect()`, `.count()`, `.show()`) is called. This allows Spark to **optimize execution plans**.

Example:

```
df = spark.read.csv('data.csv')
filtered = df.filter(df['age'] > 18) # Lazy
filtered.show() # Triggers execution
```

Interview Q&A:

Q1: What are actions vs transformations? A1:

- **Transformation:** returns a new RDD/DataFrame (lazy)
 - **Action:** triggers computation (e.g., `.collect()`, `.count()`)
-

Q2: How does lazy evaluation improve performance? A2:

- Allows **query optimization**
 - Reduces data shuffling
 - Avoids unnecessary computation
-

Q3: Can transformations be chained without execution? A3:  Yes — Spark builds a **logical execution plan** and optimizes it.

◆ 3. Broadcast Joins

Summary:

When one side of a join is **small enough to fit in memory**, it can be **broadcasted to all workers**, avoiding large shuffles.

Example:

```
from pyspark.sql.functions import broadcast

df_large = spark.read.parquet("sales.parquet")
df_small = spark.read.csv("states.csv")

joined = df_large.join(broadcast(df_small), "state")
```

Interview Q&A:

Q1: When should you use a broadcast join? A1: When one DataFrame is **small (~10MB or less)** and the other is large — to avoid shuffling.

Q2: What happens if broadcasted DataFrame is too large? A2: You may run into **OutOfMemoryError** on executor nodes.

Q3: How does Spark decide join strategy? A3:

- Automatically (via **Cost-Based Optimizer**)
 - Or manually enforced using `broadcast()`
-

4. Spark MLlib

Summary:

MLlib is Spark's **machine learning library** supporting scalable algorithms and pipelines (clustering, classification, regression, etc.).

✓ Example:

```
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.feature import VectorAssembler

df = spark.read.csv('data.csv', header=True, inferSchema=True)
vec = VectorAssembler(inputCols=["feature1", "feature2"], outputCol="features")
df = vec.transform(df)

lr = LogisticRegression(featuresCol="features", labelCol="label")
model = lr.fit(df)
```

💬 Interview Q&A:

Q1: How does Spark MLlib differ from scikit-learn? A1:

- MLlib runs **distributed**
- Pipelines are built for **scale**
- Supports streaming and batch data

Q2: What are the components of MLlib Pipeline? A2:

- **Transformer**: transforms data
- **Estimator**: fits model
- **Pipeline**: chains steps
- **Evaluator**: evaluates predictions

Q3: How do you tune hyperparameters in MLlib? A3: Use `ParamGridBuilder` + `CrossValidator` for grid search.

◆ 5. Window Functions

🔍 Summary:

Window functions allow **row-wise operations with access to neighboring rows** without collapsing the dataset — perfect for rankings, running totals, lag/lead.

✓ Example:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, desc

w = Window.partitionBy("department").orderBy(desc("salary"))
df.withColumn("rank", row_number().over(w)).show()
```

💬 Interview Q&A:

Q1: Difference between `groupBy()` and window functions? A1:

- `groupBy()` : collapses data into aggregate rows
 - **Window**: retains row-level detail while aggregating contextually
-

Q2: What are common window functions? A2: `row_number` , `rank` , `dense_rank` , `lag` , `lead` , `sum().over()`

Q3: How do you define a window? A3: Using `Window.partitionBy().orderBy()` or `Window.rowsBetween()` .

◆ 6. ETL Pipelines

🔍 Summary:

PySpark is widely used for building **scalable Extract-Transform-Load pipelines** in batch or streaming modes.

✓ Example:

```
# Extract
df = spark.read.csv("s3://bucket/raw.csv", header=True, inferSchema=True)

# Transform
df_clean = df.dropna().withColumn("year", df["date"].substr(1, 4))

# Load
df_clean.write.parquet("s3://bucket/cleaned/")
```

Interview Q&A:

Q1: How do you ensure fault-tolerant ETL in PySpark? A1: Use checkpointing, idempotent writes, error handling, and data partitioning.

Q2: What is a good practice for ETL transformation logic? A2: Use chained DataFrame APIs, avoid collecting to driver, keep all ops distributed.

Q3: Where is ETL scheduling typically done? A3: With tools like Airflow, Oozie, or Spark's built-in job scheduler.

7. Spark SQL

Summary:

Spark SQL lets you write SQL queries directly on DataFrames or temporary views — enabling **declarative** data processing.

Example:

```
df.createOrReplaceTempView("sales")
spark.sql("SELECT region, SUM(amount) FROM sales GROUP BY region").show()
```

Interview Q&A:

Q1: What's the advantage of Spark SQL over DataFrame API? A1:

- Easier for SQL-proficient users
 - Catalyst optimizer also works under the hood
-

Q2: Can you use joins, subqueries, window functions in Spark SQL? A2:  Yes — full ANSI SQL syntax supported.

Q3: What's the difference between global vs temp view? A3:

- TempView: session-scoped
 - GlobalTempView: across sessions via `global_temp.sales`
-

8. Parquet Handling

Summary:

Parquet is the most popular file format in PySpark: columnar, compressed, and splittable — perfect for big data.

Example:

```
df.write.mode("overwrite").parquet("s3://my-bucket/data/")
df_parq = spark.read.parquet("s3://my-bucket/data/")
```

Interview Q&A:

Q1: Why is Parquet preferred over CSV/JSON? A1:

- Columnar = faster column queries
- Compressed = smaller size

- Schema-aware = supports evolution
-

Q2: How does Spark optimize Parquet reads? A2:

- Predicate pushdown
 - Column pruning
 - Partition pruning
-

Q3: What happens if schema changes across files? A3: Can lead to `mergeSchema` issues — use `mergeSchema=True` or schema evolution techniques.

◆ 9. UDFs (User Defined Functions)

Summary:

A UDF allows you to use **custom Python logic** inside Spark transformations — but they **bypass Catalyst optimizations** and are **slower** than built-in functions.

Example:

```
from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType

def double(x):
    return x * 2

double_udf = udf(double, IntegerType())
df.withColumn("doubled", double_udf(df["value"])).show()
```

Interview Q&A:

Q1: When should UDFs be avoided? A1: When possible — because UDFs:

- Are not optimized by Catalyst
 - Can't leverage Tungsten codegen
 - Introduce Python-JVM serialization overhead
-

Q2: Alternatives to UDFs? A2: Use **Spark SQL functions** from `pyspark.sql.functions` or **pandas UDFs** (vectorized, faster).

Q3: What is a Pandas UDF? A3: Uses `@pandas_udf`, runs vectorized over Apache Arrow, and is **much faster** for numerical/pandas-style transformations.

◆ 10. Caching Strategies

Summary:

Use **caching** to store intermediate **DataFrames** in memory to avoid recomputation in iterative jobs.

Example:

```
df = spark.read.parquet("data.parquet").cache()
df.count() # triggers caching
```

Interview Q&A:

Q1: Difference between `cache()` and `persist()` ? A1:

- `cache()` = memory-only
 - `persist(StorageLevel.DISK_ONLY)` or others = memory + disk or disk only
-

Q2: How do you know if cache is working? A2: Use Spark UI — check if a stage's RDD/DataFrame is marked as "cached".

Q3: When should you unpersist? A3: When memory is limited or you're done with that cached DF:

```
df.unpersist()
```

◆ 11. GraphFrames

Summary:

`GraphFrames` enable **graph-parallel processing** over structured data in Spark — use cases include social networks, fraud detection, route optimization.

Example:

```
from graphframes import GraphFrame

v = spark.createDataFrame([("a",), ("b",)], ["id"])
e = spark.createDataFrame([("a", "b")], ["src", "dst"])

g = GraphFrame(v, e)
g.inDegrees.show()
```

Interview Q&A:

Q1: What Spark components does GraphFrames build on? A1: DataFrames — graphs are defined via vertex and edge tables.

Q2: Can you run BFS or PageRank in GraphFrames? A2:  Yes — GraphFrames supports:

- `bfs()`
 - `shortestPaths()`
 - `pageRank()`
 - `connectedComponents()`
-

Q3: When not to use GraphFrames? A3: For very large graphs with frequent mutation — better use GraphX in Scala or Neo4j.

◆ 12. ML Pipelines

🔍 Summary:

Spark MLlib provides **Pipeline API** similar to scikit-learn — enables chaining **transformers + estimators** for scalable ML.

✅ Example:

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.classification import LogisticRegression

assembler = VectorAssembler(inputCols=["a", "b"], outputCol="features")
scaler = StandardScaler(inputCol="features", outputCol="scaled")
lr = LogisticRegression(featuresCol="scaled", labelCol="label")

pipeline = Pipeline(stages=[assembler, scaler, lr])
model = pipeline.fit(df)
```

💬 Interview Q&A:

Q1: What are key ML pipeline stages? A1:

- **Transformers:** e.g., VectorAssembler, StandardScaler
 - **Estimators:** e.g., LogisticRegression, GBT
 - **PipelineModel:** trained pipeline
 - **Evaluator:** assess results (e.g., accuracy, AUC)
-

Q2: How do you perform hyperparameter tuning? A2: Use `CrossValidator` and `ParamGridBuilder`.

Q3: Can ML pipelines be saved and re-used? A3:  Yes — use:

```
pipeline.write().overwrite().save("path/to/save")
```

◆ 13. Partitioning

Summary:

Partitioning in Spark controls **how data is distributed across workers** — key to optimizing joins, shuffles, and parallelism.

Example:

```
# Repartition
df.repartition(10)

# Partition by column (for shuffling/join optimization)
df.repartition("region")

# Coalesce (reduce partitions without shuffle)
df.coalesce(4)
```

Interview Q&A:

Q1: What's the difference between `repartition()` and `coalesce()` ? A1:

- `repartition(n)` → shuffle to **increase** partitions
 - `coalesce(n)` → avoid shuffle to **decrease** partitions
-

Q2: When do skewed partitions hurt performance? A2: If some partitions hold **much more data**, they slow down the entire stage ("data skew").

Q3: How does Spark determine default partitioning? A3: Based on input size, source format, and Spark config (`spark.sql.shuffle.partitions`).

◆ 14. Streaming Data

Summary:

Spark Structured Streaming handles **real-time ingestion and processing** using **DataFrame API + micro-batch** model.

Example:

```
df = spark.readStream.format("kafka").option("subscribe", "topic").load()

df.writeStream \
  .outputMode("append") \
  .format("console") \
  .start()
```

Interview Q&A:

Q1: What sources are supported for streaming? A1: Kafka, socket, file systems, Kinesis, etc.

Q2: How does Spark manage streaming fault tolerance? A2: Through **checkpointing** and **write-ahead logs (WAL)**.

Q3: What is `trigger` used for in streaming? A3: To control micro-batch frequency:

```
.trigger(processingTime="10 seconds")
```

◆ 15. Scheduling

Summary:

Scheduling refers to **job execution planning**, including task distribution, stage planning, and job DAGs.

Scheduler Concepts:

- **Stages**: result from shuffles
 - **Tasks**: operate on individual partitions
 - **DAG**: directed acyclic graph of transformations
-

Interview Q&A:

Q1: How does Spark optimize job scheduling? **A1:** By converting lazy transformations into a **logical plan**, optimizing it (Catalyst), and executing as a **physical DAG**.

Q2: What causes stage boundaries? **A2:** **Shuffles**, wide transformations (e.g., `join`, `groupBy`)

Q3: How can job scheduling be monitored? **A3:** Spark UI — visualize stages, tasks, executor usage.

16. Stream Processing

Summary:

Spark's stream processing supports **stateful**, **windowed**, and **event-time** operations using SQL-like semantics.

Example:

```
from pyspark.sql.functions import window

df = df.withWatermark("timestamp", "10 minutes") \
```

```
.groupBy(window("timestamp", "5 minutes"), "user_id") \
    .count()
```

Interview Q&A:

Q1: What's the difference between event-time and processing-time? A1:

- **Event-time**: based on actual data timestamps
- **Processing-time**: based on when Spark processes the record

Q2: What is watermarking? A2: A way to **handle late-arriving data** by setting a threshold of acceptable lateness.

Q3: What is the output mode in streaming? A3:

- `append` : only new rows
- `update` : modified rows
- `complete` : full aggregate result each time

17. Batch Jobs

Summary:

Spark was originally designed for **batch processing**: ingest large datasets, apply transformations, and write out results.

Example:

```
df = spark.read.csv("s3://bucket/input/")
df_clean = df.dropna().filter(df["status"] == "active")
df_clean.write.parquet("s3://bucket/output/")
```

Interview Q&A:

Q1: How is Spark optimized for batch? A1:

- In-memory computation
 - Parallel execution
 - Catalyst optimizer + Tungsten execution engine
-

Q2: What's the difference between batch vs. streaming jobs in Spark? A2:

- Batch: finite input, one-time execution
 - Streaming: infinite input, continuous or micro-batch execution
-

Q3: How do you schedule batch jobs? A3:

- Via Apache Airflow, Oozie, cron, or cloud orchestrators (e.g., AWS Glue, Azure Data Factory)
-

18. Fault Tolerance

Summary:

Spark handles failures **automatically** using **RDD lineage**, **checkpointing**, and **retry logic**.

Fault Tolerance Techniques:

- **Lineage Graph**: recompute lost partitions
 - **Speculative Execution**: kill & retry slow tasks
 - **Checkpointing**: save RDD or stream state to persistent store
-

Interview Q&A:

Q1: How does Spark recover from executor failure? A1: Re-runs lost tasks using **lineage information** and **retry policy**.

Q2: When do you need to use checkpointing? A2:

- In **streaming jobs** with long lineage chains
- Or for **stateful transformations** like `mapGroupsWithState`

Q3: What is speculative execution in Spark? A3: Runs **duplicate copies of slow tasks** — first to finish wins. Helps mitigate straggler nodes.

Bonus Topics You Should Know

To stay ahead in real interviews, here are 4 **bonus PySpark topics** that come up often:

◆ 1. Skew Join Handling

- When one key has far more rows than others, joins become slow. Use:
 - Salting technique
 - Broadcasting small DataFrame
 - Custom partitioning

◆ 2. Delta Lake / ACID Tables

- Delta Lake adds **transactional capabilities** to Spark with schema enforcement, merge, and versioning.

◆ 3. Job DAG Analysis

- Understanding **Spark UI**, DAGs, stages, and tasks is crucial for tuning performance.

◆ 4. Adaptive Query Execution (AQE)

- Spark 3+ can **dynamically adjust join strategy**, number of partitions, and plans based on runtime stats.

Perfect — here's the full Q&A format for the first 5 coding questions

◆ 1. Select only 'name' and 'age' columns

Q1: How do you select specific columns in a PySpark DataFrame?

A1: Use `.select()` to return only the desired columns.

```
df.select("name", "age").show()
```

💡 This is equivalent to `SELECT name, age FROM table`.

◆ 2. Filter rows where age > 30

Q2: How do you filter rows based on a condition?

A2: Use `.filter()` or `.where()` with the condition inside.

```
df.filter(df.age > 30).show()
```

💡 Returns only rows where age is greater than 30.

◆ 3. Fill missing ages with the average age

Q3: How do you replace nulls in a column with the average value?

A3: Use `.fillna()` after calculating the mean.

```
from pyspark.sql.functions import avg

mean_age = df.select(avg("age")).collect()[0][0]
df_filled = df.fillna({"age": mean_age})
df_filled.show()
```

💡 `collect()` gets the average from a single-row DF, and `fillna()` replaces nulls.

◆ 4. Add column 'age_group': 'young' if age < 30, else 'old'

Q4: How can you add a column with conditional logic?

A4: Use `when()` and `otherwise()` from `pyspark.sql.functions`.

```
from pyspark.sql.functions import when

df.withColumn("age_group", when(df.age < 30, "young").otherwise("old")).show()
```

💡 `when-otherwise` is Spark's version of `if-else`.

◆ 5. Group by 'dept' and calculate average salary

Q5: How do you group data and apply an aggregation?

A5: Use `.groupBy()` and `.agg()` or shortcut functions like `.avg()`.

```
from pyspark.sql.functions import avg

df.groupBy("dept").agg(avg("salary").alias("avg_salary")).show()
```

💡 This gives the average salary per department.

◆ 6. Sort DataFrame by salary descending

Q6: How do you sort a PySpark DataFrame in descending order?

A6: Use `.orderBy()` with `desc()`.

```
from pyspark.sql.functions import desc

df.orderBy(desc("salary")).show()
```

💡 You can also sort ascending (default) or by multiple columns.

◆ 7. Join with another DataFrame to get department locations

Q7: How do you perform an inner join between two DataFrames?

A7: Use `.join()` with join column and mode.

```
df_joined = df.join(df_join, on="dept", how="inner")
df_joined.select("name", "dept", "location").show()
```

💡 PySpark supports joins: `inner`, `left`, `right`, `outer`, etc.

◆ 8. Drop rows with missing scores

Q8: How do you remove rows that have `null` in a specific column?

A8: Use `.dropna()` with `subset`.

```
df_missing_cleaned = df_missing.dropna(subset=["score"])
df_missing_cleaned.show()
```

💡 Without `subset`, it drops rows with null in **any** column.

◆ 9. Count number of 'fail' status values

Q9: How do you count rows with a specific column value?

A9: Use `.filter()` followed by `.count()`.

```
df_missing.filter(df_missing.status == "fail").count()
```

💡 You can also group and count with `.groupBy("status").count()`.

◆ 10. Rename column 'salary' to 'income'

Q10: How do you rename a column in a DataFrame?

A10: Use `.withColumnRenamed()`.

```
df_renamed = df.withColumnRenamed("salary", "income")
df_renamed.show()
```

💡 This doesn't rename in-place; it returns a new DataFrame.

◆ 11. Create a column 'tax' = 10% of salary

Q11: How do you add a derived column in PySpark?

A11: Use `.withColumn()` along with arithmetic operations.

```
from pyspark.sql.functions import col

df.withColumn("tax", col("salary") * 0.10).show()
```

💡 `col("salary") * 0.10` computes the tax value row-wise.

◆ 12. Get top 2 highest paid employees

Q12: How do you fetch the top N rows based on a column?

A12: Use `.orderBy(desc(...)).limit(N)`.

```
df.orderBy(col("salary").desc()).limit(2).show()
```

💡 Efficient for leaderboard-style queries.

◆ 13. Calculate cumulative sum of salary

Q13: How do you calculate a running total (cumulative sum)?

A13: Use `Window` function with `rowsBetween`.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum
```

```
window_spec = Window.orderBy("id").rowsBetween(Window.unboundedPreceding, 0)
df.withColumn("cumulative_salary", sum("salary").over(window_spec)).show()
```

💡 Use `orderBy()` for proper cumulative logic.

◆ 14. Replace 'Finance' with 'FinOps' in 'dept' column

Q14: How do you replace values in a column conditionally?

A14: Use `when().otherwise()` .

```
from pyspark.sql.functions import when

df.withColumn("dept", when(col("dept") == "Finance", "FinOps").otherwise(col("dept"))).show()
```

💡 This is non-destructive and returns a new DataFrame.

◆ 15. Get distinct departments

Q15: How do you get unique values of a column?

A15: Use `.select().distinct()` .

```
df.select("dept").distinct().show()
```

💡 Equivalent to SQL's `SELECT DISTINCT dept` .
